Programación Avanzada

Tema 3. Herencia, clases abstractas y polimorfismo

Índice

Esquema

Ideas clave

3.1. Introducción y objetivos

3.2. Relación de generalización y especialización entre clases

3.3. Principio de sustitución de Liskov

3.4. Herencia

3.5. Clases abstractas

3.6. Polimorfismo

3.7. Ligadura de métodos

3.8. Cuaderno de ejercicios

A fondo

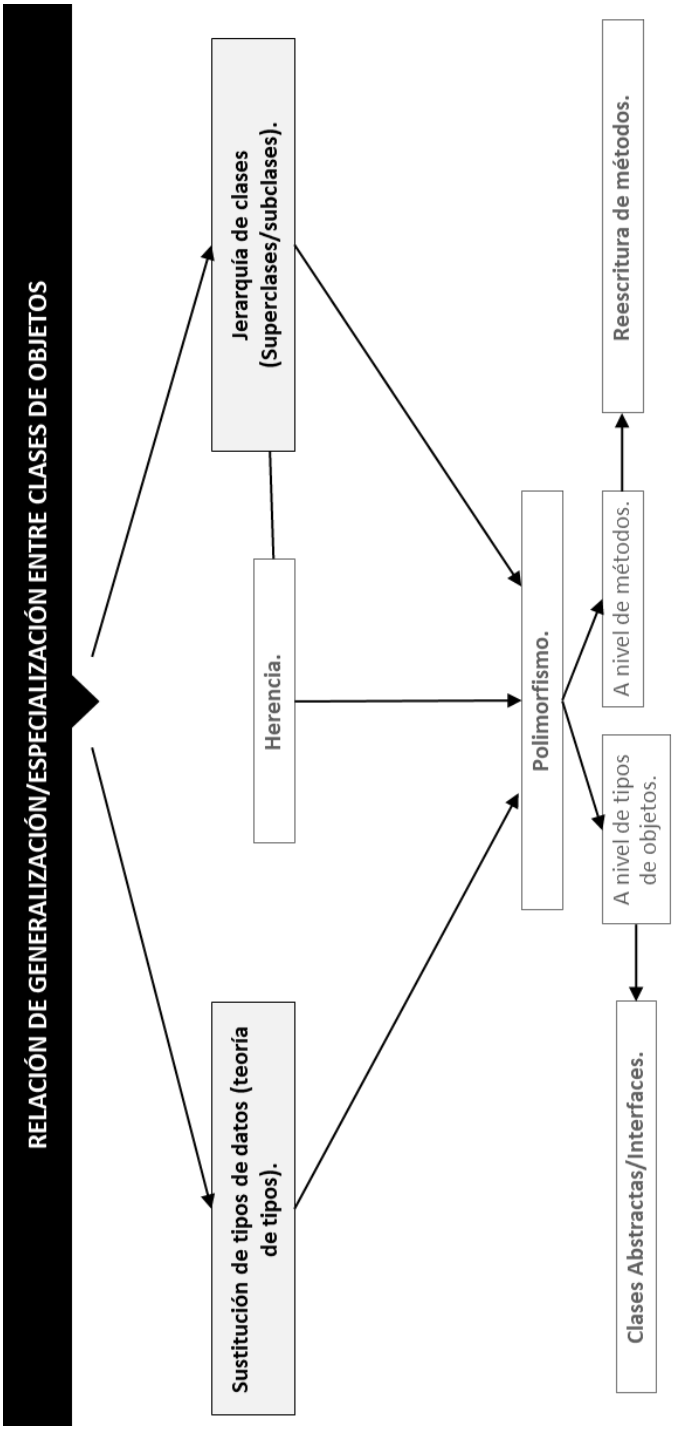
A Behavioral Notion of Subtyping by Barbara H. Liskov , Jeanette M. Wing

Enlace estático y dinámico en los lenguajes de programación. | | UPV

Java Programming Masterclass for Software Developers

Test

Esquema



© Universidad Internacional de La Rioja (UNIR)

3.1. Introducción y objetivos

El paradigma de orientación a objetos modela el dominio de un problema de programación como un conjunto de clases de objetos colaborativos. De todas las relaciones existentes entre estos objetos, existe una que tiene implicaciones muy importantes en el diseño, como es la **relación de generalización/especialización de clases.**

La importancia de la relación de generalización/especialización es que en ella subyace un mecanismo muy potente de reutilización y estructuración de código denominado herencia. La reutilización es la base del diseño y la construcción de muchos sistemas modernos, y se articula típicamente mediante *frameworks* prediseñados que, al reutilizarlos, permiten alcanzar cotas de productividad y calidad considerables.

Además de la reutilización, la herencia permite implementar una característica fundamental, como es el polimorfismo. Mediante el polimorfismo se puede implementar un código flexible ante cambios y enviar un mismo mensaje a objetos que se comportan de manera diferente como veremos más adelante en este tema.

Los **objetivos generales** de este tema pueden ser resumidos en los siguientes:

- Estudiar y comprender la relación de generalización/especialización.
- Comprender las implicaciones de la herencia dentro del diseño de clases.
- Estudiar el uso de clases e interfaces abstractas.
- Aprender a implementar una jerarquía de clases.
- Comprender el polimorfismo.

© Universidad Internacional de La Rioja (UNIR)

3.2. Relación de generalización y especialización entre clases

La relación de generalización se presenta cuando una entidad conceptual A representa a una generalización de otra más específica: B. La relación entre A y B responde a la semántica **B es un A o B es un tipo de A.**

Una superclase en una relación de generalización representa todo lo que es abstractamente equivalente a sus subclases.

En la Figura 1 se puede ver un ejemplo de la generalización del concepto figura geométrica (Forma). La relación establecida entre el concepto general y los conceptos generalizados tendrá, como veremos, implicaciones estructurales en la definición de las clases.

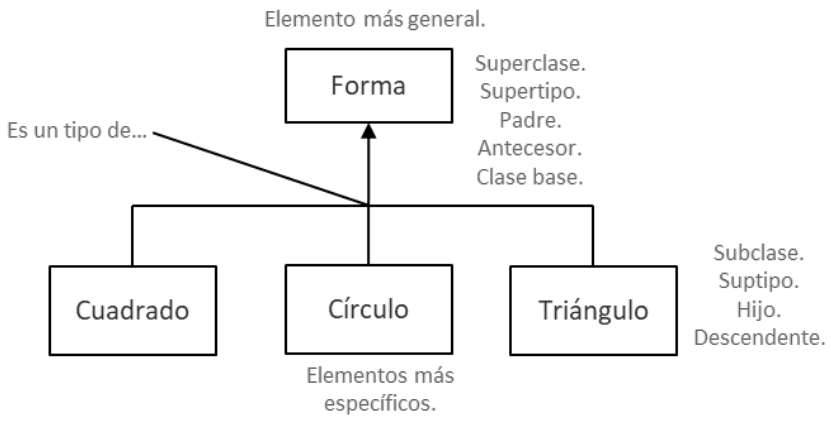


Figura 1. Representación de una relación de generalización. Fuente: elaboración propia.

En el esquema anterior, podemos ver diferentes nombres comúnmente usados para las clases que participan en una relación de generalización. Los nombres más comunes para la clase general son: **superclase, clase base, clase padre**. Para las clases específicas son: **subclase, subtipo, hijo, descendiente.**

© Universidad Internacional de La Rioja (UNIR)

Mediante esta generalización conceptual (Forma) podemos reunir lo que es común a cualquier forma geométrica (en términos de clases, serían datos y comportamiento) y hacer que cualquier forma geométrica nueva sea un subtipo o subclase de esta generalización. Luego, la generalización organiza estructuralmente a un conjunto de clases.

Para ver las implicaciones de este tipo de relación en diseño orientado a objetos, es importante que recordemos la definición de tipo de datos:

Un tipo de dato describe la representación, interpretación y estructura de los valores de una variable

Desde el punto de vista de **la interpretación de las subclases,** no debe olvidarse que su representación, interpretación y estructura de valores (su tipo) **debe hacerse en términos de la clase base y su estructura propia.**

En modelado de clases, existen dos situaciones por las que podríamos llegar a esta relación de jerarquía: a partir de un proceso de especialización desde el concepto general o, dado una serie de especializaciones existentes, generalizando a un concepto general.

De manera general, podemos definir la relación de generalización como **un tipo de relación en la que una clase** (clase general) **generaliza el comportamiento y las propiedades de otras clases** (subclases). De otra manera, desde el punto de vista de las subclases, podemos decir que las subclases especializan el comportamiento de la clase general.

© Universidad Internacional de La Rioja (UNIR)

La forma de representar una relación de generalización en UML es la siguiente:

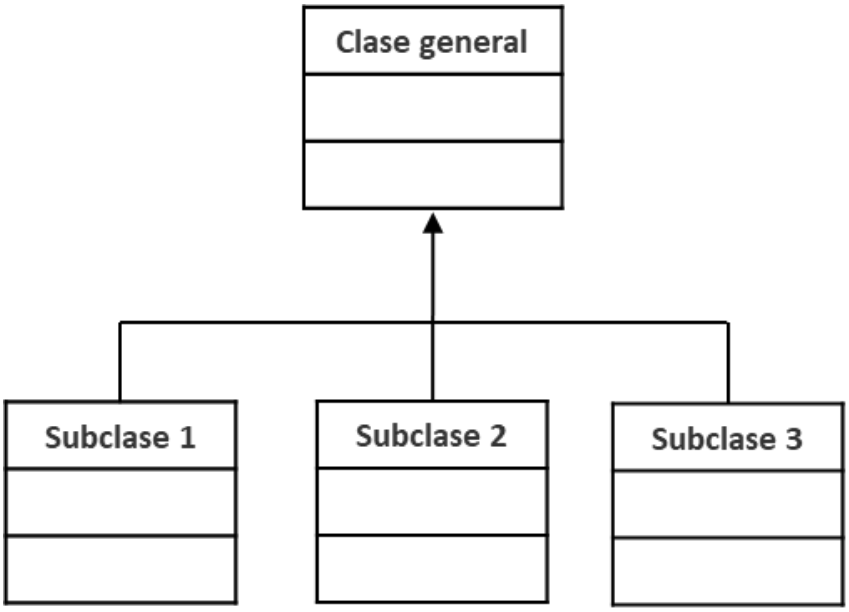


Figura 2. Generalización/especialización en notación UML. Fuente: elaboración propia.

Desde el punto de vista estructural, **cada subclase incluirá el comportamiento y las características de la superclase, salvo que los redefina o los especializase en la propia subclase.** La subclase, por tanto, tendrá la opción de dejar el comportamiento de la clase base de la que es un tipo o redefinir un comportamiento más específico.

Una subclase es una versión extendida y especializada de su superclase.

© Universidad Internacional de La Rioja (UNIR)

A continuación, se muestran un ejemplo de este tipo de relación, donde la entidad Publicación generaliza al concepto de Revista, Libro y Acta.

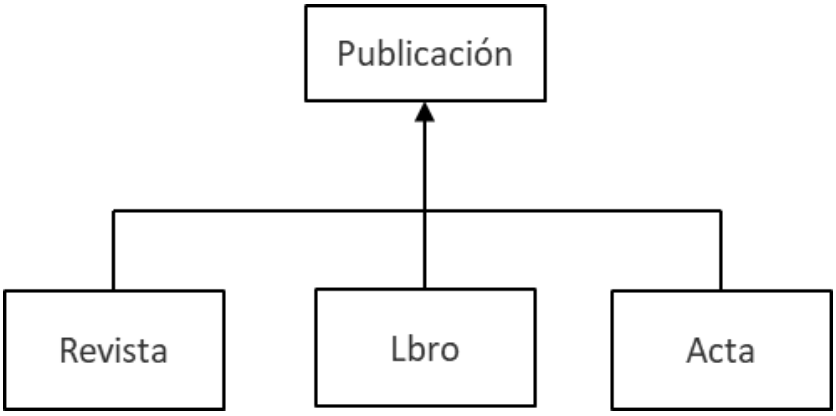


Figura 3. Ejemplo de relación de generalización. Fuente: elaboración propia.

Al extender una clase, se heredan todas las operaciones de la padre y se puede añadir nuevas operaciones.

A continuación, se muestra un resumen de la visibilidad dentro de una jerarquíaa de herencia en función de la visibilidad indicada en los atributos:

Visibilidad	Comienza por	Desde la propia clase	Desde las subclases	Desde otras clases
- privado	__	✓	✗	✗
# protegido	_	✓	✓	✗
+ público		✓	✓	✓

Figura 4. Resumen de la visibilidad. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

3.3. Principio de sustitución de Liskov

¿Cómo saber que la jerarquía de clases está bien hecha? ¿Cómo saber que el diseño de la jerarquía va a preservar las propiedades de la clase base y que no se van a vulnerar mediante la redefinición en las subclases? Estas cuestiones hacen que sea necesario establecer una serie de restricciones que deben cumplirse y que garanticen que la jerarquía satisface los criterios de diseño.

El principio de Liskov se presenta de la siguiente forma:

> Si S es un subtipo de T, entonces los objetos de tipo T en un programa de computadora pueden ser sustituidos por objetos de tipo S (es decir, los objetos de tipo S pueden sustituir objetos de tipo T), sin alterar ninguna de las propiedades deseables de ese programa (la corrección, la tarea que realiza, etc.).

El **principio de sustitución de Liskov** (introducido inicialmente por Barbara Liskov) **es una definición particular de una relación de subtipificación, llamada tipificación (fuerte) del comportamiento.**

Este principio **garantiza la interoperabilidad semántica de tipos en una jerarquía de clases.** O, dicho de otra forma, este principio **determina un criterio de corrección de una jerarquía de tipos.**

Liskov y Jeannette Wing formularon el principio de manera conjunta en un artículo en el año 1994 de la siguiente manera**: dada la siguiente asignación x: T:= E.**

© Universidad Internacional de La Rioja (UNIR)

El requisito para el subtipo E es: sea p una propiedad demostrable sobre los objetos x de tipo T, entonces p debería ser verdadera para objetos y del tipo E, donde E es un subtipo de T.

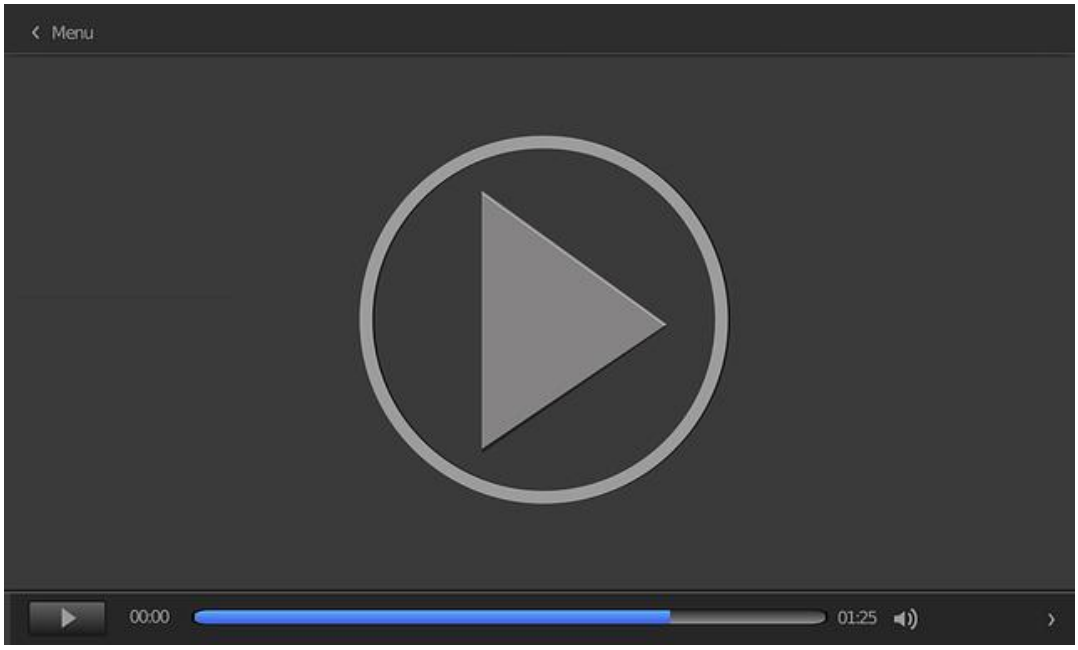
© Universidad Internacional de La Rioja (UNIR)

3.4. Herencia

La herencia constituye una de las **propiedades más importante de la orientación a objetos** (aunque existen lenguajes orientados a objetos que no tienen herencia). **Mediante ella se implementa la reutilización código y la sustitución de tipos.**

La herencia permite que una clase sea un subtipo (subclase o clase derivada) de otra clase (su superclase o clase base).

A continuación, puedes ver el vídeo *La reutilización del software.* El video muestra las características del software y las técnicas empleadas para alcanzarlo.



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=52496544-5532-4295-a8f5-aff600eda830

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Cuando se crea una jerarquía de clases, implícitamente, existe una relación de herencia entre las clases que participan en la relación, porque las subclases heredan todas las características de sus superclases (un objeto instancia de una subclase es también una instancia de la superclase). Para ser más específicos, **en relación de herencia, las subclases heredan lo siguiente** de la clase base:

- Atributos.
- Operaciones.
- Relaciones.
- Restricciones.

Posteriormente, cada una de las subclases podrá añadir nuevas características o podrá reescribir las operaciones de la superclase para hacerlas especificas a su comportamiento si se requiere.

El comportamiento de las subclases queda determinado por la clase base de la que heredan y por el comportamiento especifico que definen. La subclase, por tanto, podrá añadir a su diseño tantos atributos y métodos como considere, siempre y cuando no vulnere las restricciones impuestas por el principio de sustitución Liskov visto anteriormente.

© Universidad Internacional de La Rioja (UNIR)

3.5. Clases abstractas

Las relaciones de generalización introducen un nivel conceptual de clases que, más que clases de implementación, son muchas veces conceptos abstractos de organización para las subclases.

Una clase abstracta es una clase genérica (o tipo de objeto) que se utiliza como base para crear objetos específicos que se ajustan a su protocolo o al conjunto de operaciones que admite.

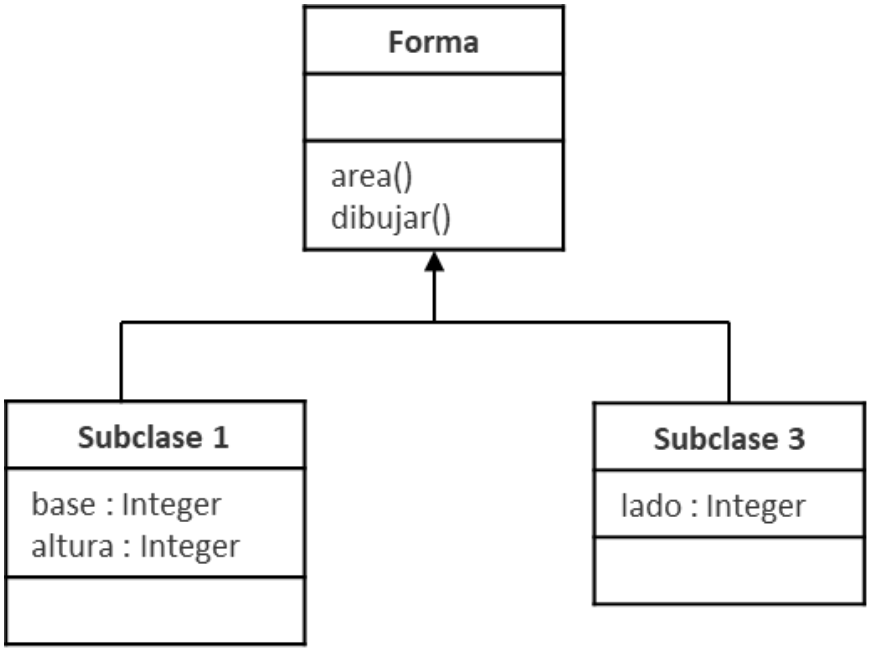


Figura 5. Clases abstractas. Fuente: elaboración propia.

En nuestro ejemplo, la operación **dibujar (…)** de la clase Forma es un buen ejemplo, porque no sabríamos como pintar una Forma abstracta sin estar en un nivel de Forma específica (Círculo o Cuadrado). Se dice, entonces, que una clase abstracta tienen operaciones o un comportamiento abstracto. Podemos indicar esto haciendo que el método de la clase sea abstracto.

© Universidad Internacional de La Rioja (UNIR)

Una clase que no puede dar una implementación a alguno de sus métodos se denomina clase abstracta. Una clase con al menos uno de sus métodos abstractos será una clase abstracta.

Debido a que las **clases generales con métodos abstractos** no tienen un comportamiento definido (es decir, no tienen una implementación para sus métodos), **estas clases no pueden ser instanciadas.**

Una clase abstracta deberá definir al menos un método que sea abstracto, pero puede tener métodos normales.

Una clase abstracta es una clase restringida que no se puede utilizar para crear objetos (para acceder a ella se debe heredar de otra clase). Una clase abstracta no puede ser instanciada. Un método abstracto solo se puede usar en una clase abstracta y no tiene cuerpo. El cuerpo lo proporciona la subclase.

En la Figura 6, tenemos una clase abstracta llamada Forma, con dos métodos abstractos **dibujar(…)** y **obtenerArea()**. La implementación para estas operaciones las proporciona tanto la subclase Circulo como la subclase Cuadrado. Aunque la clase abstracta Forma es incompleta, y no puede ser instanciada, sus subclases si son completas cuando son instanciadas.

© Universidad Internacional de La Rioja (UNIR)

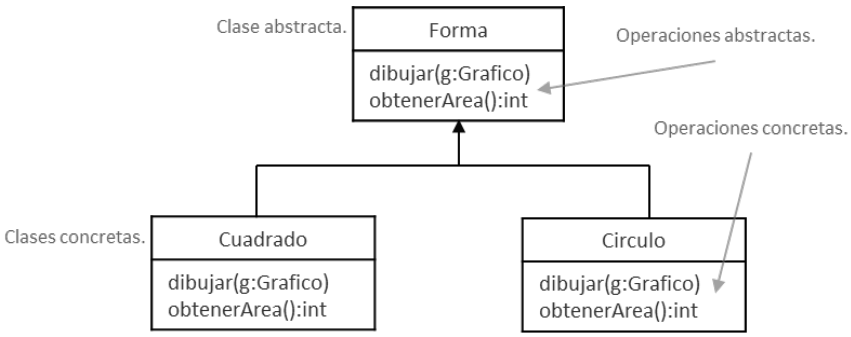


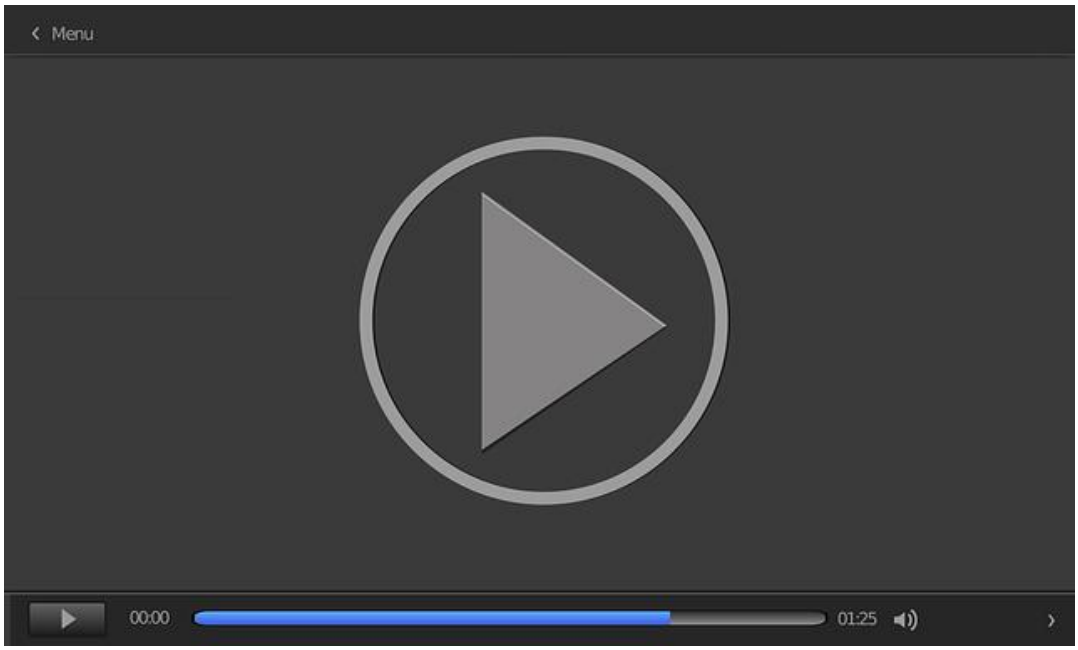
Figura 6. Clases abstracta y clases concretas. Fuente: elaboración propia.

El propósito de introducir clases abstractas puede resumirse en las siguientes **ventajas:**

- **Se pueden definir un conjunto de operaciones abstractas,** en la clase abstracta de Forma, que están obligadas a ser implementadas por las subclases. Se puede ver esto como un contrato que todas las subclases deben cumplir.
- Se puede proporcionar un código que opere con cualquier subtipo de manera uniforme por medio de la sustitución de tipos, lo que hace a el código más flexible.

© Universidad Internacional de La Rioja (UNIR)

Te dejamos el vídeo *Interfaces abstractas.* Las interfaces abstractas constituyen una herramienta fundamental a la hora de introducir puntos de desacoplamiento en el diseño y de invertir dependencias. Este video muestra las interfaces abstractas desde esta perspectiva.



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=cf5617df-dfff-4354-9a5f-aff600ef7b11

© Universidad Internacional de La Rioja (UNIR)

3.6. Polimorfismo

En lenguajes de programación y teoría de tipos, el polimorfismo se refiere al uso de una sola interfaz para tipos diferentes o el uso de un único símbolo para representar varios tipos diferentes.

Si asumimos que una clase representa un tipo de dato, las clases que participan en una relación de herencia definen tipos y subtipos según su posición en la jerarquía. **Por el principio de sustitución**, se pueden usar objetos pertenecientes a un subtipo en lugares donde se espera un objeto del supertipo. El subtipo es intercambiable allí donde aparezca una expresión con el supertipo. Este permite que haya operaciones que se definan en términos del supertipo y admitan objetos subtipos.

Es importante resaltar que la sustitución va de abajo arriba, es decir, se permite sustituir al supertipo por el subtipo pero no al revés.

Una variable de un supertipo puede apuntar a un objeto del supertipo (si se puede crear ese tipo de objeto) y a cualquier subtipo descendiente en la relación jerárquica.

Al tomar el contexto de una clase, lo que permite el polimorfismo es que **se puedan manejar subclases distintas con un mismo identificador.** Al usar este identificador común, se pueden invocar operaciones denominadas polimórficas, de forma que el envió de un mismo mensaje pueda resolverse con comportamientos diferentes.

En el ejemplo de la jerarquía de Formas geométricas, el método area() es un claro ejemplo de operación polimórfica. El método area() es común a toda interfaz de las clases pertenecientes a la relación de generalización, pero en cada nivel tiene un comportamiento distinto.

© Universidad Internacional de La Rioja (UNIR)

Una operación polimórfica es aquella que tiene varias implementaciones.

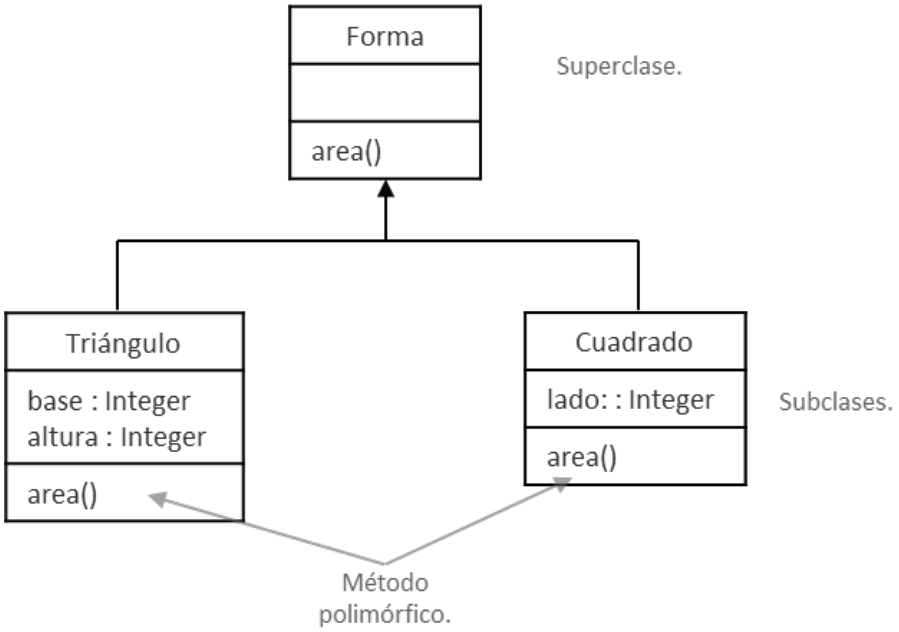


Figura 7. Operaciones polimórficas. Fuente: elaboración propia.

Las subclases Triángulo y Cuadrado heredan de la superclase Forma y proporcionan una implementación específica y particular para la método **area().** Luego, el método **area()** tiene diferentes formas como método (es un método polimórfico) dentro de la relación de jerarquía. El tipo de dato (superclase) Forma también es polimórfico porque puede tomar la forma de un Triángulo y de un Cuadrado.

El **polimorfismo** se basa en **dos propiedades importantes:**

- **Referencias a objetos polimórficos:** una referencia a una superclase puede apuntar a un objeto de cualquiera de sus subclases.
- **Operaciones polimórficas:** la operación invocada se selecciona con base en la clase del objeto, no en la de la referencia.

© Universidad Internacional de La Rioja (UNIR)

Vamos a ver cómo funciona el polimorfismo con un ejemplo de utilización: supongamos que queremos calcular el área total cubierta por una serie de figuras. Un problema surge de inmediato: el tipo de figuras es variable, es decir, podemos tener dos Cuadrados o dos Triángulos y, *a priori,* esto no está determinado en el tiempo de compilación. **Aquí es donde el polimorfismo presenta todo su potencial.** El siguiente ejemplo (Figura 8) muestra como el código para calcular el área total se puede plantear en términos del supertipo para poder **manejar la variabilidad existente en la generalización** y resolver el cálculo del área en el momento de la invocación (llamada a **método polimórfico**), según sea el tipo de figura instanciada en ese momento.

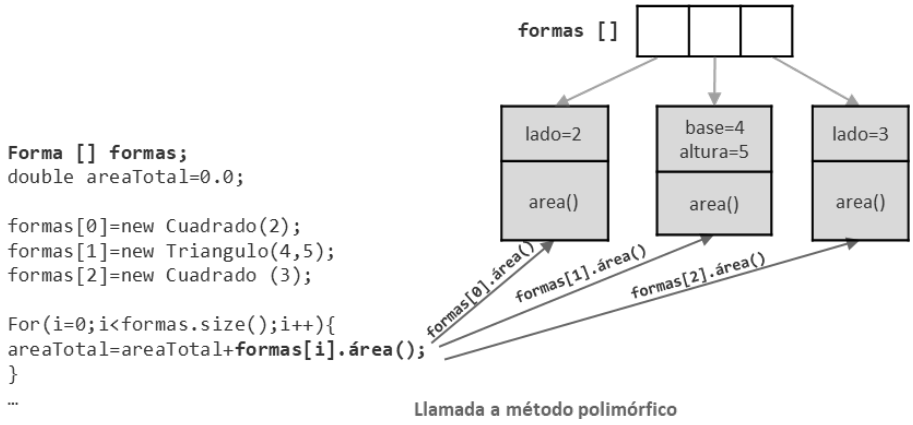


Figura 8. Llamada a método polimórfico para manejar la variabilidad de subtipos. Fuente: elaboración propia.

Claramente, la implementación **area()** diferirá para cada subclase. Esta es la **esencia del polimorfismo, objetos de diferentes clases tienen operaciones con el mismo prototipo (igual signatura), pero diferentes implementaciones.**

Hemos dicho que **el polimorfismo permite abordar la variabilidad existente en una jerarquía de tipos** ¿Qué pasa si en nuestra aplicación introducimos un nuevo tipo de figura? (Figura 9)

© Universidad Internacional de La Rioja (UNIR)

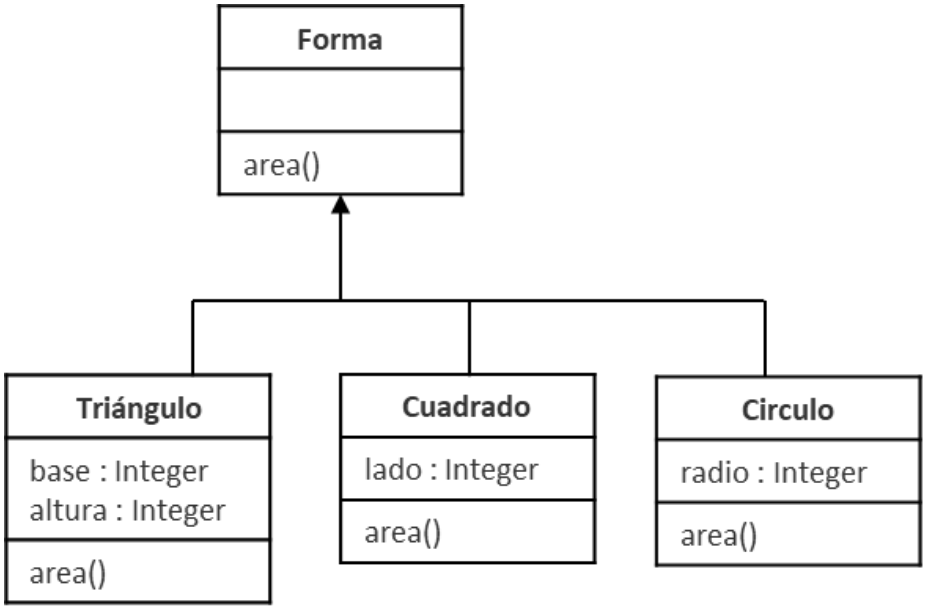


Figura 9. Ampliación de un nuevo tipo de forma. Fuente: elaboración propia.

La respuesta es nada, como se puede en la siguiente figura (Figura 10), la llamada a nivel de método se resuelve igual que las demás, y nivel de tipos es un tipo polimórfico.

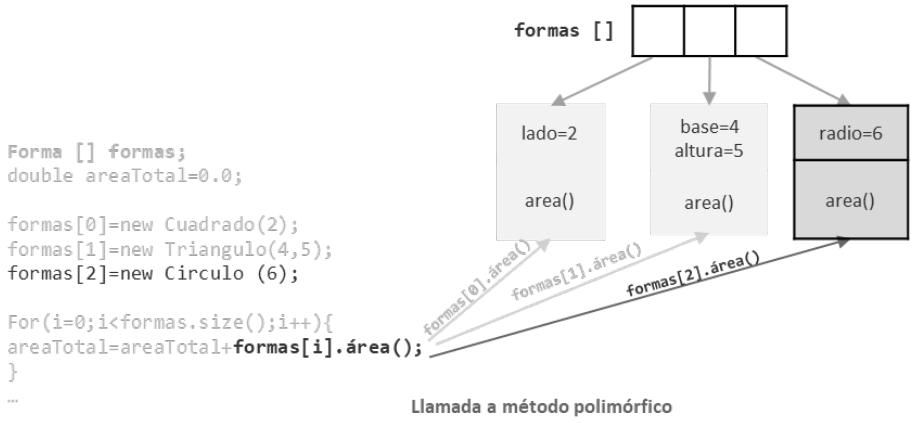


Figura 10. Introducción de un nuevo tipo de forma. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Como vemos, el polimorfismo es una propiedad muy importante en programación orientada a objetos y puede verse desde distintos puntos de vista:

- Capacidad de un lenguaje de programación para manejar varios tipos de objetos con una interfaz uniforme.
- Capacidad de diferentes objetos para responder al mismo mensaje de una manera específica.

El **polimorfismo** nos permite diseñar sistemas más simples, que **se adaptan mejor a los cambios** porque permite tratar objetos diferentes de la misma forma. Esto permite añadir flexibilidad al código que interactúa con una clase polimórfica porque esta podría ser sustituida por otra que ofreciese el mismo comportamiento **(preservación de comportamiento)**, pero con distinta implementación.

Como vemos, existen diferentes tipos de polimorfismo.

La construcción del lenguaje que hace posible el polimorfismo es la ligadura dinámica —conocida también como ligadura tardía o postergada— entre llamadas a funciones y los cuerpos reales de dichas funciones. En cualquier caso, cada invocación a **dibujar()** para cada uno de los objetos de tipo forma será resuelto (en el tiempo de ejecución) invocando la versión especifica de **dibujar()** para cada uno de los subtipos.

Redefinición de métodos en subclases

Los métodos de una clase pueden redefinirse de diferentes formas para ser polimórficos.

- Sobreescritura de métodos heredados en una subclase.
- Sobreescritura de métodos propios (no depende de la herencia). Polimorfismo paramétrico, también llamada sobrecarga de métodos.

© Universidad Internacional de La Rioja (UNIR)

Sobreescritura de métodos heredados (Overriding)

Cuando una clase pertenece a una relación de generalización como subclase de otra, puede ser interesante cambiar el comportamiento heredado.

Sea la Clase A que define el método **m** y sea la Clase B una subclase de A y que hereda **m**, la clase B puede optar por las siguientes opciones de diseño sobre el método **m**:

- **Heredar el método m de A y no redefinirlo:** el comportamiento del método **m** es heredado por la Clase B.
- **Sobrescribir o redefinir el método m de A:** el comportamiento de **m** es especializado por la clase B y **m** se convierte en un método polimórfico.

En el ejemplo mostrado en la siguiente figura, las subclases Triangulo y Cuadrado heredan todos los atributos, operaciones y restricciones de la superclase Forma.

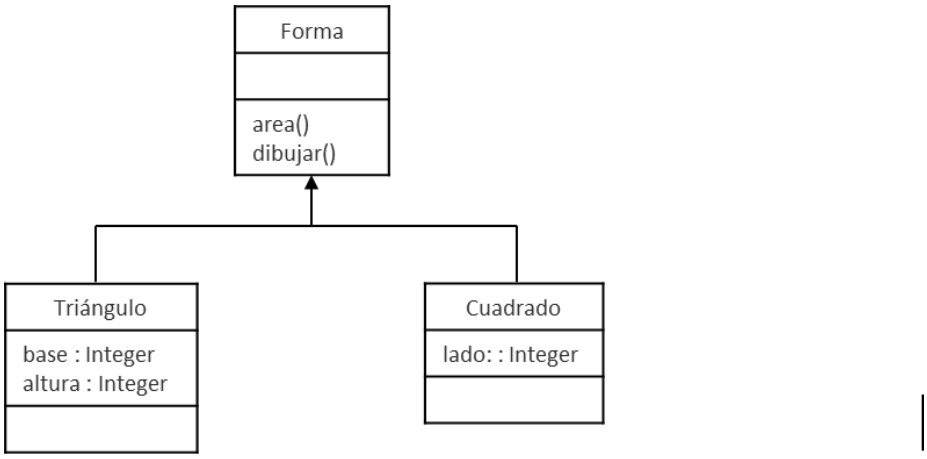


Figura 11. Jerarquía de clases. Fuente: elaboración propia.

La clase Forma define el método **area().** La definición de estos métodos obedece a que por diseño queremos que toda Forma presente este comportamiento. Pero no se puede definir estos métodos a este nivel.

© Universidad Internacional de La Rioja (UNIR)

La siguiente figura muestra esta acción: las subclases Triángulo y Cuadrado proporcionan sus propias operaciones para el método **area() .**

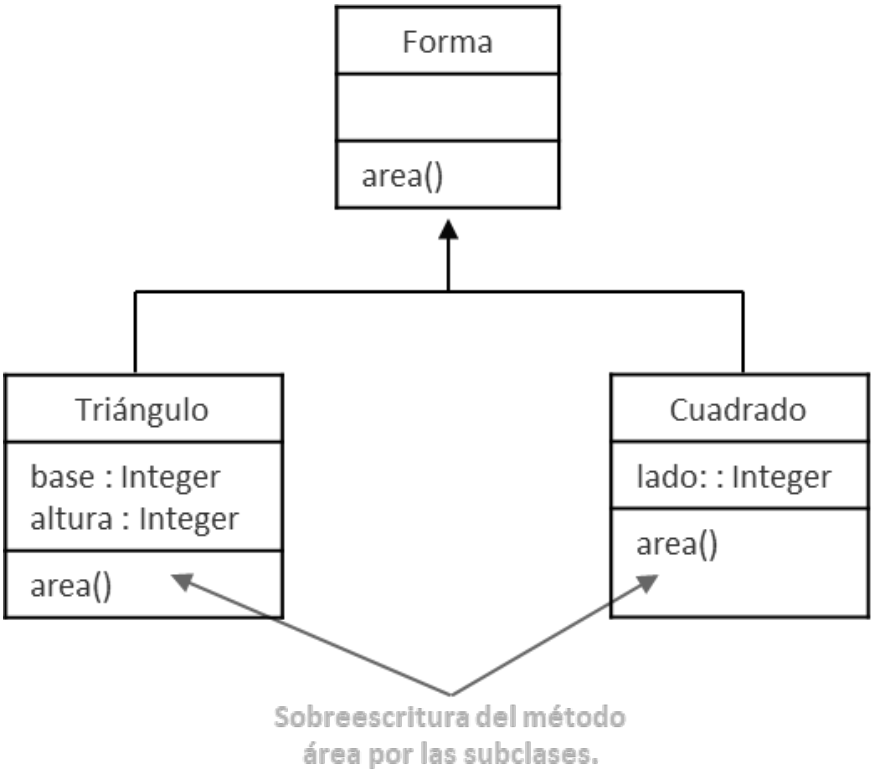


Figura 12. Reescritura de métodos heredados. Fuente: elaboración propia.

Para reescribir una operación de la clase base, una subclase debe proporcionar una operación con exactamente el mismo prototipo de la operación de la clase base.

Implementación de la reescritura de métodos

A continuación, se muestra como implementar la reescritura de los métodos abstractos que define la clase forma:

```
public abstract class Forma {

public abstract double area();

}
```


© Universidad Internacional de La Rioja (UNIR)

```
public class Triangulo extends Forma {

private final double base;

private final double altura;

public Triangulo (double base, double altura)

{

this.base = base; this.altura = altura;

}

@Override

double area() { return (base / 2) * altura; }

}
```

Sobrecarga de métodos (overloading)

Por otra parte, en la definición de una clase, un método se puede definir con distintos números de parámetros. A esta operación se la denomina **sobrecarga del método.**

En el siguiente ejemplo, podemos ver como el método **suma** es un método sobrecargado en la definición de la clase Calculadora:

```
public Class Calculadora {

public int sumar ( int a,int b){ return a+b;}

public int sumar (int a,int b,int c) {return a+b+c;}

}
```


© Universidad Internacional de La Rioja (UNIR)

Las siguientes llamadas al método **sumar** de la clase Calculadora son perfectamente válidas:

```
s.sumar(2,4);

s.sumar(2,4,8);
```


© Universidad Internacional de La Rioja (UNIR)

3.7. Ligadura de métodos

Los símbolos empleados en el lenguaje de programación requieren de una realización y correspondencia con mecanismos internos de representación dentro de la máquina.

La ligadura **representa,** generalmente, una **conexión entre una entidad y sus propiedades.** Si la propiedad se limita a funciones, la ligadura es la **conexión entre la llamada a una función y el código que se ejecuta** tras la llamada.

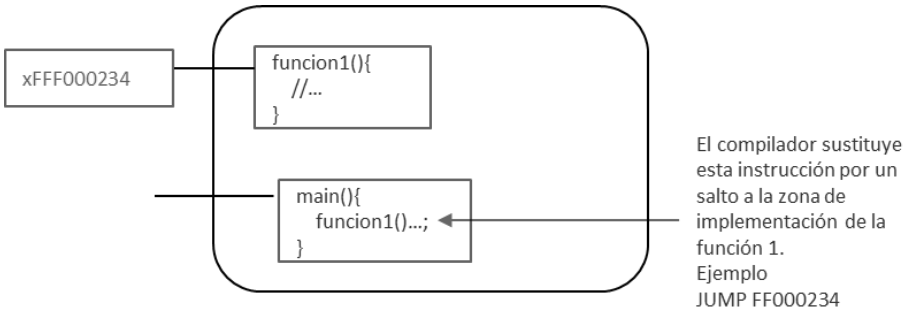


Figura 13. Ligadura de métodos. Fuente: elaboración propia.

El momento de tiempo en el que un atributo o función se asocia con sus valores o funciones se denomina **tiempo de ligadura.**

La ligadura se clasifica según sea el tiempo o momento del enlace, y esta puede ser **estática o dinámica.**

- **La ligadura estáticase produce antes de la ejecución (en tiempo de compilación).** En un lenguaje de programación con ligadura estática, todas las referencias se determinan en tiempo de compilación. La mayoría de los lenguajes procedimentales son de ligadura estática; el compilador y el enlazador *(linker)* definen directamente la posición fija del código que se va a ejecutar en cada llamada a la función. Aunque está posición es relativa al segmento donde finalmente se ubicará la función en memoria.

© Universidad Internacional de La Rioja (UNIR)

- La ligadura dinámica supone que el código a ejecutar en respuesta a un mensaje no se determinará hasta el momento de la ejecución. Únicamente la ejecución del programa (normalmente un puntero a la clase base) determinará la ligadura efectiva entre las diversas que son posibles (una para cada clase derivada).

La ligadura dinámica se encarga de ligar o relacionar la llamada a un método con el cuerpo del método que se ejecuta finalmente.

Vamos a considerar que la notación virtual sobre un método de una clase indica al compilador que esa función será implementada en una subclase, no en la clase donde se define. Vamos a suponer que sólo se puede usar esta opción cuando exista la posibilidad de que una subclase implemente los métodos denominados virtuales.

© Universidad Internacional de La Rioja (UNIR)

Vamos a imaginarnos que en un instante de tiempo dado en la memoria hay tres objetos instancia de tres clases diferentes:

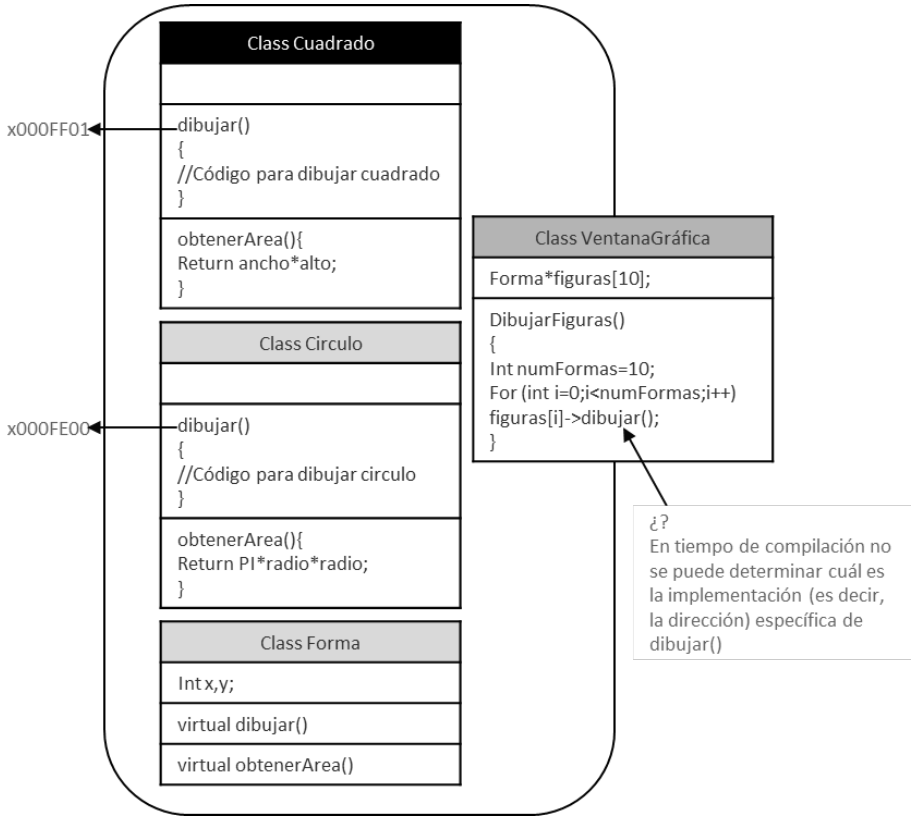


Figura 14. Mapa de memoria, funciones virtuales y polimorfismo. Fuente: elaboración propia.

La cuestión es ¿Cómo resuelve el compilador la siguiente instrucción?

```
figuras[i]->dibujar();
```

Donde figuras[i] es una *array* de objeto de tipo Figura.

© Universidad Internacional de La Rioja (UNIR)

El compilador tiene que enlazar la llamada a la función **dibujar()** con alguna zona de memoria del objeto correspondiente , pero, como vemos en este caso, en la Figura 14 no tiene ninguna implementación. Pero, como está indicado con la palabra reservada **virtual**, el compilador sabe que debe buscar esta implementación en alguna clase derivada. ¿Cuál de ellas? La que en ese momento este en la posición indexada.

Ventajas de la ligadura dinámica

La principal ventaja de la ligadura dinámica frente a la ligadura estática es que la dinámica ofrece un alto grado de flexibilidad y diversas ventajas prácticas para manejar jerarquías de clases de un modo muy simpe. Entre las desventajas de la ligadura dinámica, se encuentra que —en principio— es menos eficiente que la ligadura estática.

Los lenguajes orientados a objetos que sí siguen estrictamente este paradigma ofrecen solo ligadura dinámica. Los lenguajes híbridos (C++,Simula), ofrecen los dos tipos de ligadura.

© Universidad Internacional de La Rioja (UNIR)

3.8. Cuaderno de ejercicios

1. Represente las siguientes relaciones de generalización

- Una colección de elementos puede ser una lista enlazada, un árbol binario o un conjunto.
- Un tipo de dato numérico puede ser de tipo byte (8 bits), entero corto (16 bits), entero largo (64 bits) , entero (32 bits) ,real (32 bits) y real de precisión doble (64 bits)

Solución

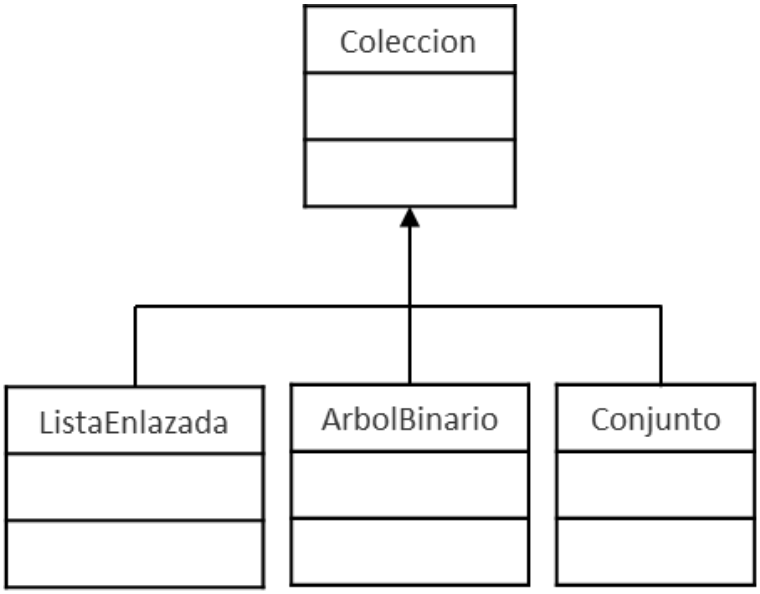


Figura 15. Ejercicio 1 a. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

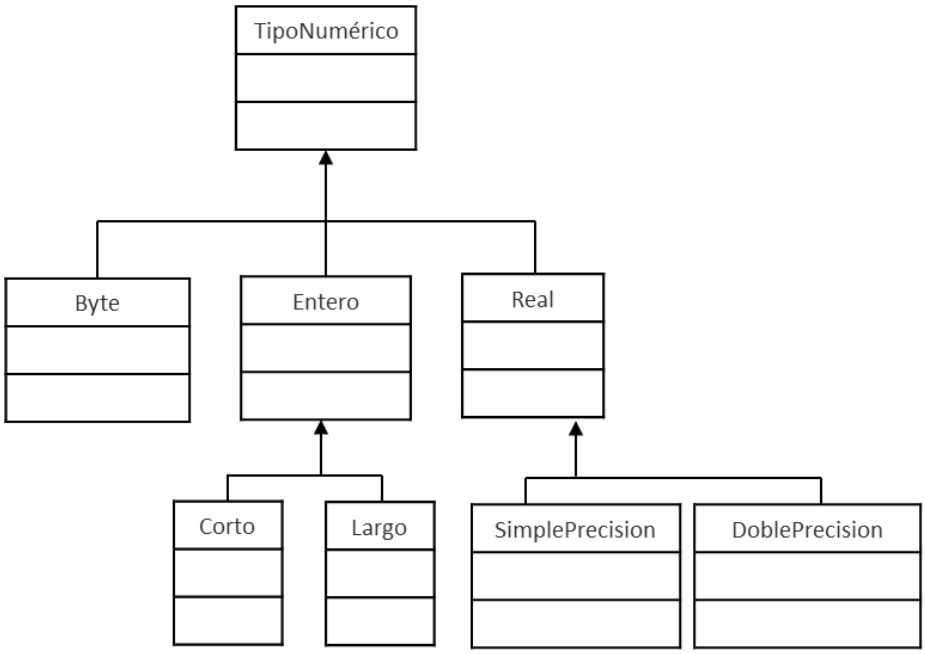


Figura 16. Ejercicio 1 b. Fuente: elaboración propia.

2. Dado el siguiente código:

```
Shape s1 = new Circulo(5);

System.out.println(s1);

System.out.println(s1.area());



Circle c1 = (Circle)s1;

System.out.println(c1);

System.out.println(c1.Area());
```

- ¿Qué imprime por pantalla la línea **System.out.println(s1.area());**?
- ¿Es valida la instrucción **Circle c1 = (Circle)s1;**?

© Universidad Internacional de La Rioja (UNIR)

Solución

- Imprime el área del círculo de radio 5 area=($\pi 5^2$)
- Sí, es válida. Se hace una operación de *downcasting* de tipo válida.

3. Un diagrama UML permiten describir un sistema desde distintas perspectivas. Los diagramas, en general, pueden ser un diagrama estructural o un diagrama de comportamiento. Los diagramas estructurales son de clases, de componentes o de objetos. Los diagramas dinámicos, por otra parte, pueden ser de actividad, de estados, de interacción o de casos de uso. Los diagramas de interacción son importantes porque, a su vez, pueden ser de comunicación y de secuencia.

Realice un modelo de clases UML del dominio anterior.

Posible solución

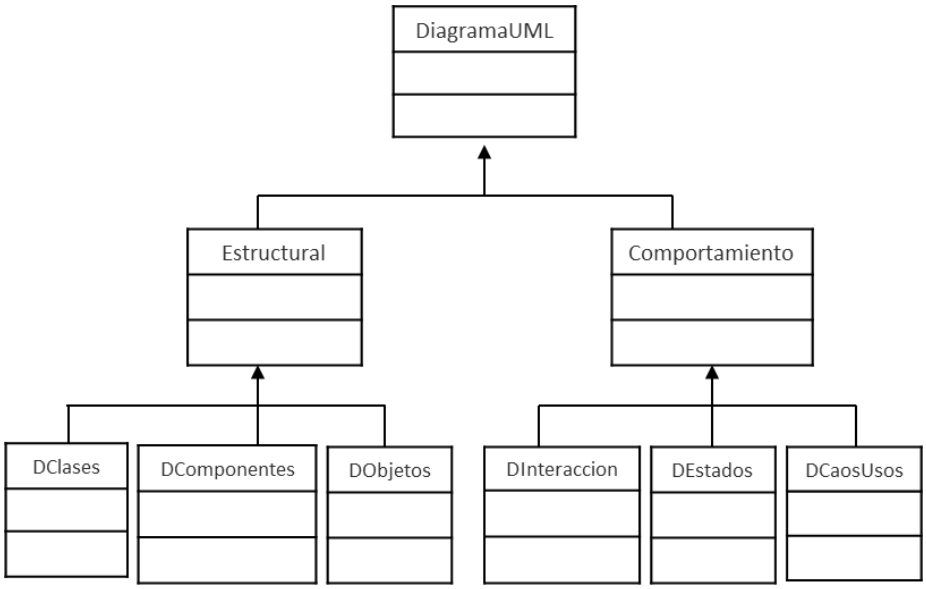


Figura 17. Ejercicio 3. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

4. La clase Thread de Java representa un hilo de ejecución. Una clase puede comportarse como un hilo si implementa la interfaz Runnable.

```
public interface Runnable{

public void run()

}

public class Tarea implements Runnable {

public void run(){// implementación de la tarea}

}
```

Cuestiones: ¿Es el método **run()** dentro de la definición de Tarea polimórfico?

Solución

Sí el método **run** es polimórfico porque la clase sobrescribe un método de una clase abstracta a la cual está ligado.

5. Observa el siguiente fragmento de código:

```
if(userInput.equals("student"))

s2 = new Student(…);

else

s2 = new GradStudent(…);
```

Responde a las siguientes cuestiones:

- ¿Qué tipo de objeto tendrá s2 en la ejecución del programa si **user Input** tiene cadena "student"?

© Universidad Internacional de La Rioja (UNIR)

- ¿El programa está usando ligadura dinámica o estática? Razona su respuesta.

Solución

- S2 tendrá el tipo Student.
- Está usando ligadura dinámica porque el tipo de dato no se conoce en tiempo de compilación.

Dado el siguiente modelo de clases en UML:

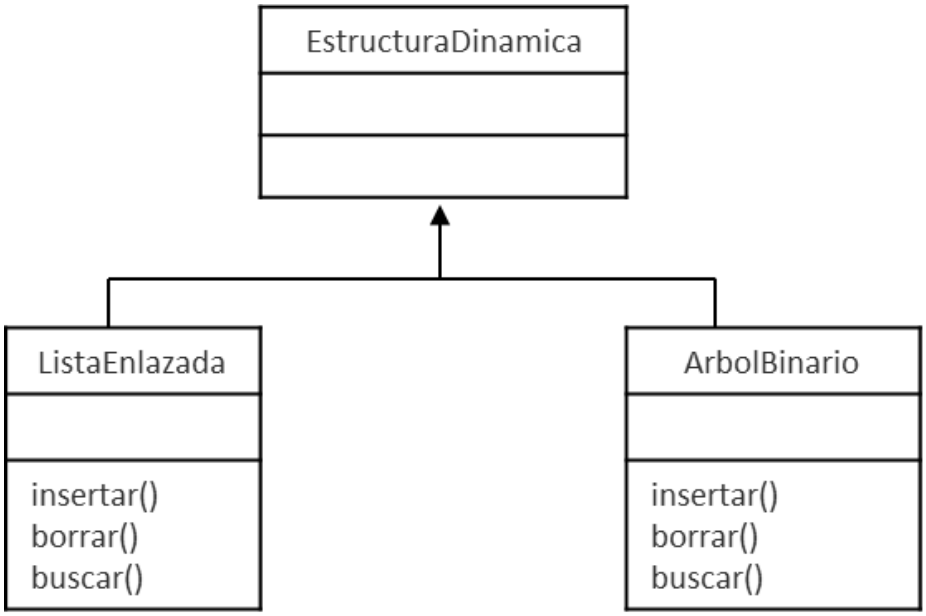


Figura 18. Ejercicio 6. Fuente: elaboración propia.

6. Responde a las siguientes pregunta:

- ¿Los métodos insertar, borrar y buscar son polimórficos?
- ¿Una EstructuraDinámica según este modelo es polimórfica?
- ¿Una aplicación que trabajase con este modelo de estructuras dinámicas podría intercambiar una lista enlazada por un árbol?

© Universidad Internacional de La Rioja (UNIR)

Solución

- ¿Los métodos insertar, borrar y buscar son polimórficas? Sí, son los mismos métodos con distintas implementaciones.
- ¿Las estructuras dinámicas según este modelo son polimórficas? Sí, puede ser una ListaEnlazada o un ArbolBinario.
- ¿Una aplicación que trabajase con este modelo de estructuras dinámicas podría intercambiar una lista enlazada por un árbol? Sí, podría.

7. Añada al modelo anterior listas doblemente enlazadas.

Una posible solución sería

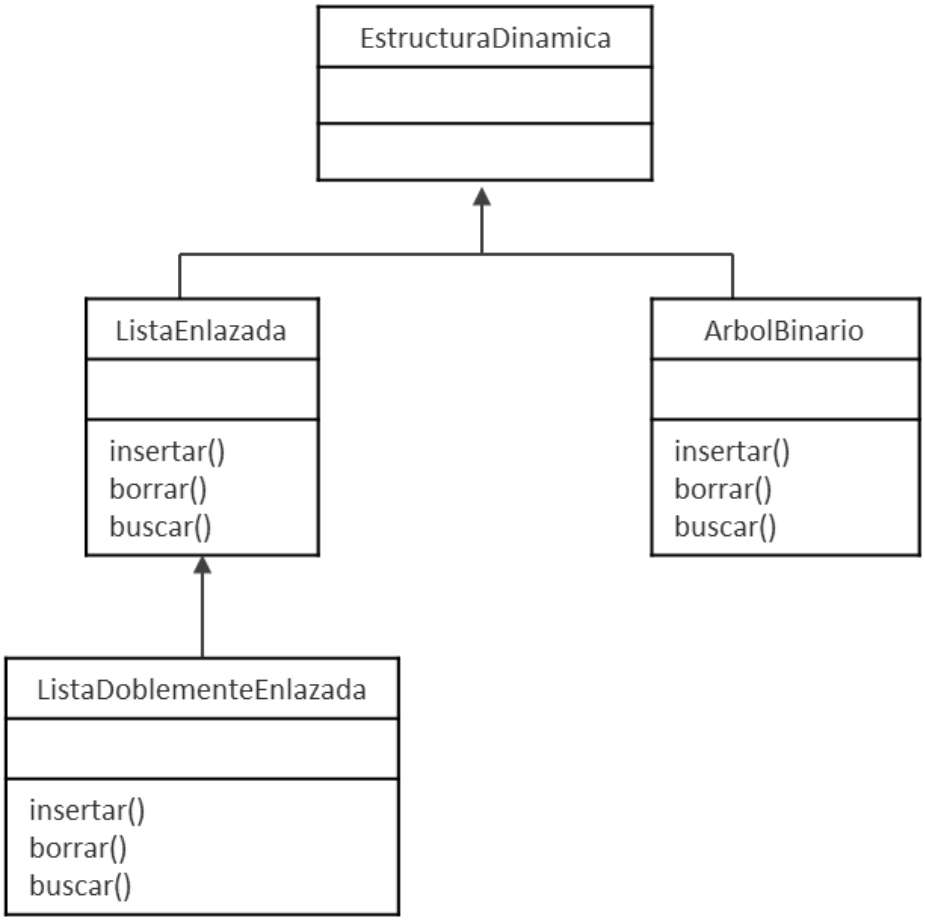


Figura 19. Ejercicio 7. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

8. Un clasificador de imágenes puede ser de aprendizaje supervisado y no supervisado. Dentro de los de aprendizaje supervisados se encuentran los que usan métodos estadísticos y los basados en redes neuronales. Por otra parte, los clasificadores de aprendizaje no supervisados también se clasifican en aquellos que usan métodos estadísticos y basados en redes neuronales.

Diseñe un modelo de clases que represente los diferentes tipos de clasificadores.

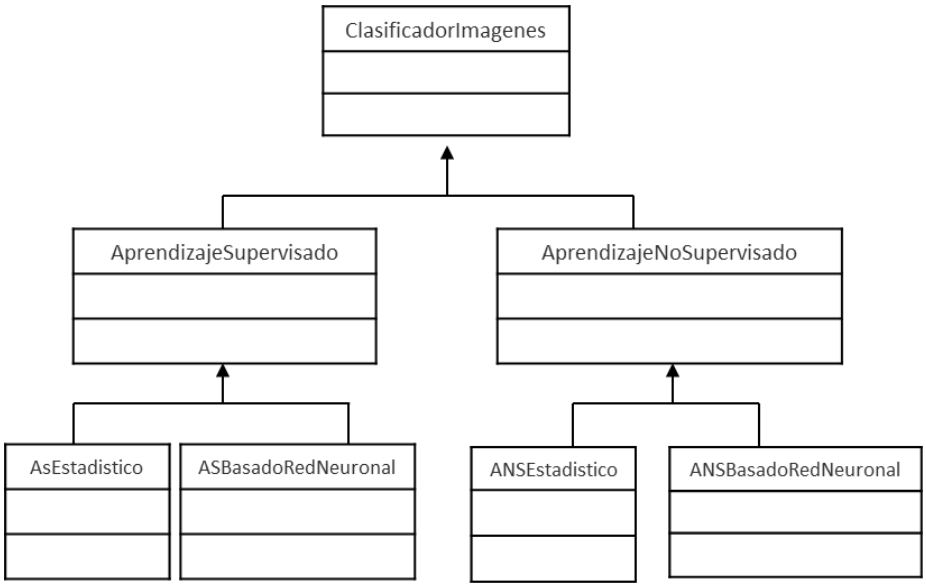


Figura 20. Ejercicio 8. Fuente: elaboración propia.

9. Dada la siguiente clase Almacen

```
public class Almacen{

private int N=10;

private DiscorSSD [N]discos_ssd;

private DiscosMecanicos [N]discos_mecanicos;
```


© Universidad Internacional de La Rioja (UNIR)

```
public int totalCapacidadDiscos () {

int acum1,acum2;

for ( int i=0;i< N;i++){

if discossd[i]!=null

acum1= acum1+discos_ssd.capacidad();

}

for ( int i=0;i< N;i++){

if discosmecanicos[i]!=null

acum2= acum2+discos_mecanicos.tam();

}

retrun acum1+acum2;

}
```

Se pide: cambie el diseño de la clase Almacen para que sume el total de capacidad de todos los discos a partir de un único tipo de discos usando un tipo polimórfico.

© Universidad Internacional de La Rioja (UNIR)

Una posible solución sería

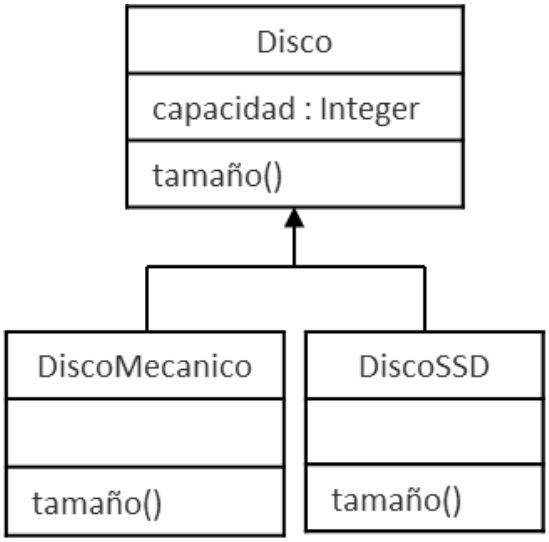


Figura 21. Ejercicio 9. Fuente: elaboración propia.

```
public class Almacen{

private int N=10;

private Discos [N]discos;

public int totalItemsInventario () {

item acum;

for ( int i=0;i< N;i++){

acum =acum+discos[i].tamaño();

}

return acum;

}
```


© Universidad Internacional de La Rioja (UNIR)

10. Codifica las siguientes clases junto con sus constructores

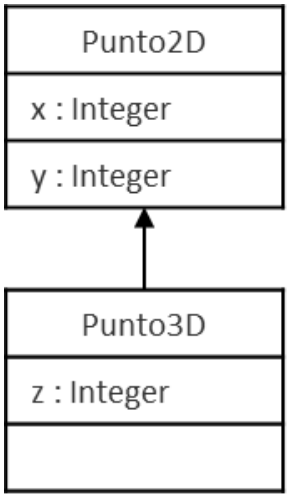


Figura 22. Ejercicio 10. Fuente: elaboración propia.

Posible solución

```
public class Punto2D{

private double x;

private double y;

public Punto 2D( double x,double y){

this.x=x;

this=y=y;

}

}

public class Punto 3D{

Private double z;
```


© Universidad Internacional de La Rioja (UNIR)

```
Public Punto3D ( double x,double ,y, double z){

super(x,y);

this.z=z;

}

}
```


© Universidad Internacional de La Rioja (UNIR)

A Behavioral Notion of Subtyping by Barbara H. Liskov , Jeanette M. Wing

Liskov, B. H. (1994). *A Behavioral Notion of Subtyping.* ACM Transactions on Programming Languages and Systems. http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.39.1223

¿Qué significado tiene el que un tipo sea subtipo de otro? El uso de la jerarquía de tipos es una parte muy importante en el diseño orientado a objetos. La jerarquía de tipos permite el uso de familias de tipos, en las que los supertipos de nivel superior capturan el comportamiento que todos sus subtipos tienen en común. Para que esta característica de diseño sea eficaz, es necesario tener una comprensión clara de cómo se relacionan los subtipos y los supertipos. Este importante artículo define los requisitos y restricciones para que la relación asegure que cualquier propiedad probada sobre los objetos de supertipo también sea válida para sus objetos de subtipo.

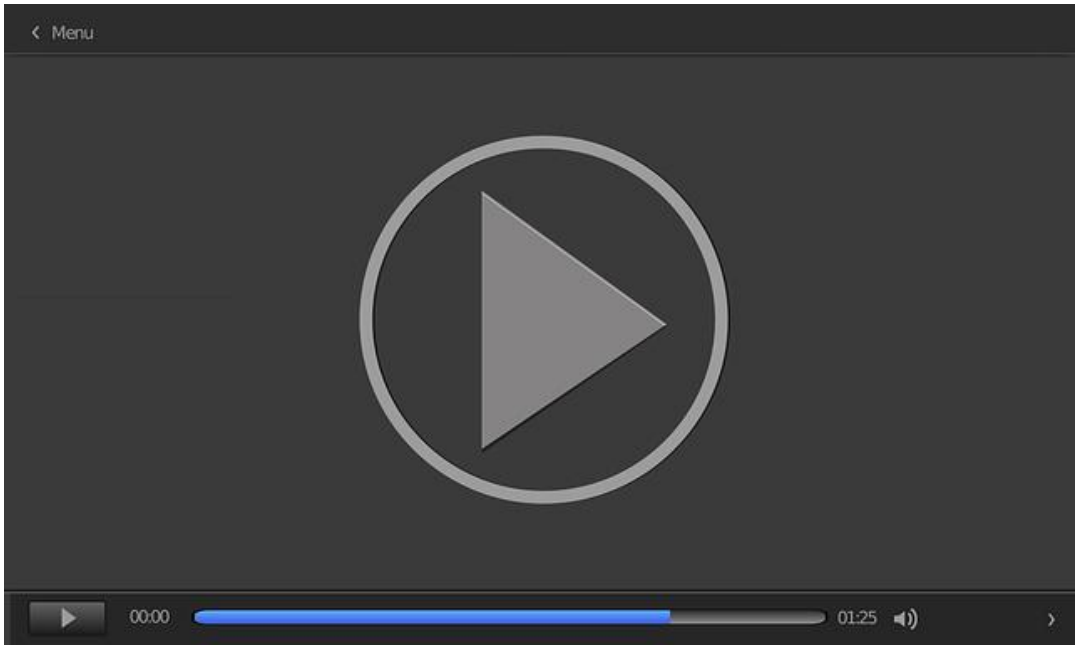
© Universidad Internacional de La Rioja (UNIR)

Enlace estático y dinámico en los lenguajes de programación. | | UPV

Universitat Politècnica de València – UPV. (2017, octubre 19). *Enlace estático y dinámico en los lenguajes de programación. || UPV.* [Vídeo]. YouTube. https://www.youtube.com/watch?v=y2eCjadS8x8

El tipo enlace de las llamadas a métodos y su representación en la memoria determina la forma de ejecución de los métodos, además de una serie de propiedades importantes relacionadas. En esta sesión podrás ver la diferencia entre enlace estático y dinámico, para ello se plantea la problemática de trabajar con un enlace estático y uno dinámico en un lenguaje de programación como es C# y se explica cómo diferenciar dichos enlaces y cómo se puede trabajar con ellos.

© Universidad Internacional de La Rioja (UNIR)



Accede al vídeo:

https://www.youtube.com/embed/y2eCjadS8x8

© Universidad Internacional de La Rioja (UNIR)

Java Programming Masterclass for Software Developers

Buchalka, T. (2021). *Polymorphism*. [Vídeo]. Udemy. https://www.udemy.com/tutorial/java-the-complete-java-developer-course/polymorphism/

Excelente video que muestra cómo funciona el polimorfismo. Observa cuando crea la función random para mostrar como la instancia del objeto que se va a usar no puede estar determinada en tiempo de compilación.

© Universidad Internacional de La Rioja (UNIR)

Test

1. El principio de sustitución de Liskov restringe a que:

   A. Todo subtipo debe preservar las propiedades del supertipo.

   B. Toda característica del supertipo debe preservarse en el subtipo.

   C. Todo comportamiento del supertipo debe preservarse en el subtipo.

   D. Todas las anteriores son correctas.

2. Señala la respuesta incorrecta con respecto al concepto de polimorfismo:

   A. Se puede presentar a nivel de método.

   B. Se puede presentar a nivel de tipo de objeto.

   C. Se presenta únicamente en una jerarquía de clases.

   D. Requiere un cambio de nombre en el método polimórfico.

3. La ligadura de un método con su llamada puede ser:

   A. Constante o variable.

   B. Dinámica o estática.

   C. Temporal o permanente.

   D. Directa o indirecta.

4. Indique que afirmación de las siguientes es falsa:

   A. La herencia se produce de la clase base hacia las subclases.

   B. La herencia se produce desde las subclases a la clase base.

   C. La herencia se da a nivel de atributos solamente.

   D. La herencia se da a nivel de métodos únicamente.

© Universidad Internacional de La Rioja (UNIR)

5. Sobre la reescritura de métodos, indique la afirmación incorrecta:

A. El comportamiento de una clase base se puede redefinir en la subclase.

B. Los métodos de una subclase pueden ser redefinidos solo si son abstractos.

C. Los métodos abstractos de una clase base deben ser implementados obligatoriamente en las subclases.

D. Los atributos de una superclase son heredados por una subclase.

6. Respecto al polimorfismo, indique la respuesta correcta:

A. Un método polimórfico es aquel que presenta el mismo nombre, pero con diferentes parámetros.

B. Un método polimórfico es aquel que presenta un mismo prototipo de función, pero con distintas implementaciones.

C. Un método polimórfico debe ser abstracto en la clase base y redefinido al menos por una subclase.

D. Todas las anteriores son verdaderas.

7. Para que sirve la sobrecarga de métodos:

A. Para ofrecer una misma operación o método con distintos parámetros.

B. Para ofrecer una misma operación implementada de forma distinta en distintas clases.

C. Para ofrecer una misma operación con idéntico número de parámetros con distinta implementación.

D. Para implementar subclases.

© Universidad Internacional de La Rioja (UNIR)

8. Respecto a las clases abstractas indique la correcta:

A. Una clase abstracta define al menos un método abstracto.

B. Una clase abstracta define todos sus métodos abstractos obligatoriamente.

C. Una clase abstracta puede instanciarse mediante ligadura estática.

D. Todas son incorrectas.

9. Indique la respuesta incorrecta:

A. Una subclase presenta un alto acoplamiento respecto a la clase base.

B. Una subclase presenta un bajo acoplamiento respecto a la clase base.

C. Una subclase puede redefinir el comportamiento de la clase base.

D. Una subclase puede implementar métodos polimórficos.

10. ¿Qué hace que B sea subclase de A?

A. Que B tenga algo en común con A.

B. Que B tenga las propiedades de A.

C. Que B pueda sustituirse allí donde aparece A.

D. Todas las respuestas anteriores son correctas.

© Universidad Internacional de La Rioja (UNIR)